

Machine, Data and Learning

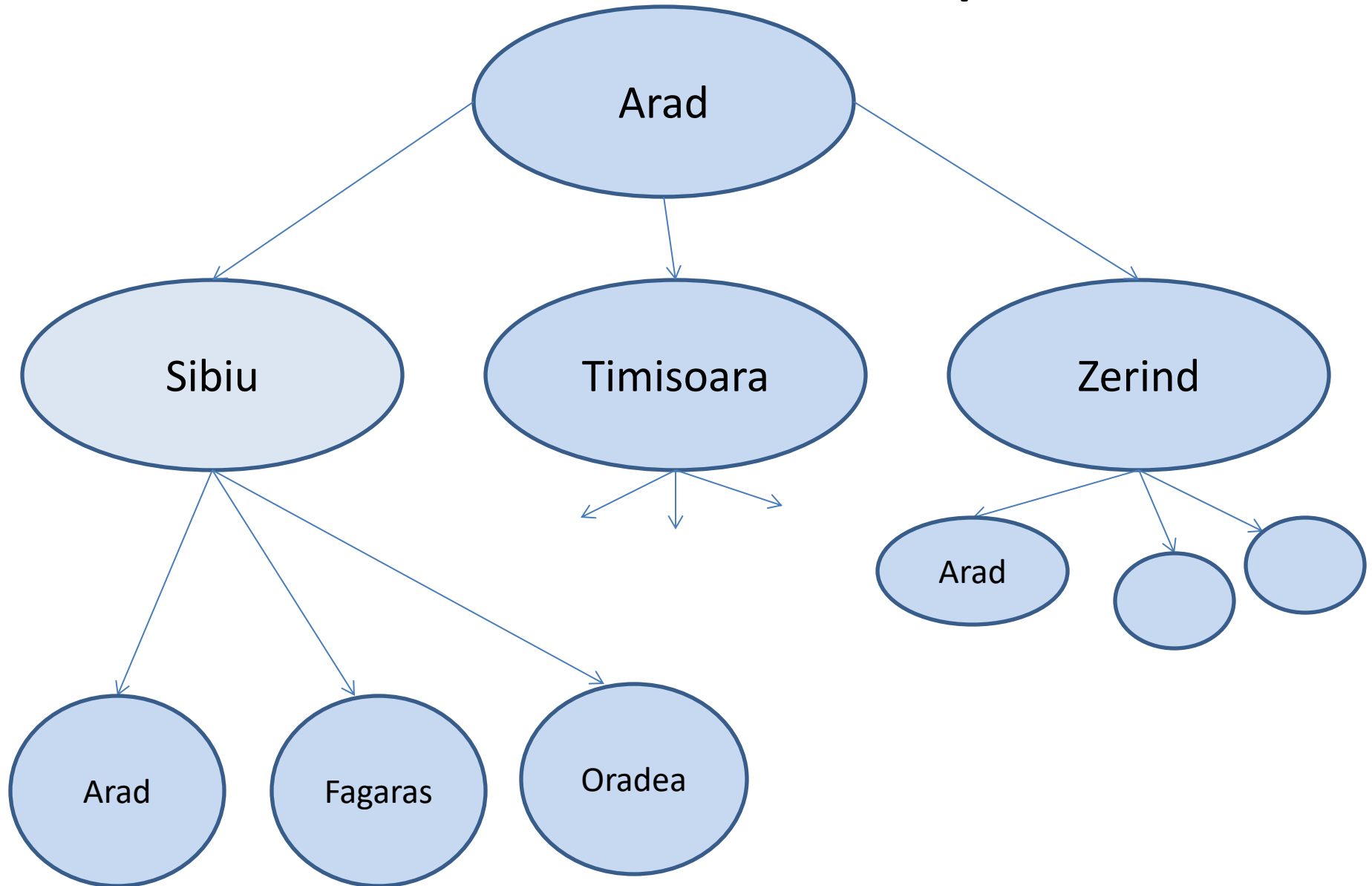
Search

- Process of looking for a **sequence of actions** that reaches the goal
- Search algorithm takes problem as input and outputs action sequence
 - Agent executes these actions
- Agent has to drive from point 1 to 2 in a map : Here environment is **observable** (agent knows current state), **discrete** (finite action set), **known** (knows states reached by actions) and **deterministic** (each action has one outcome)
 - Solution here is a fixed sequence of actions
 - Agent can ignore percepts in this case !!! (**Why ???**)

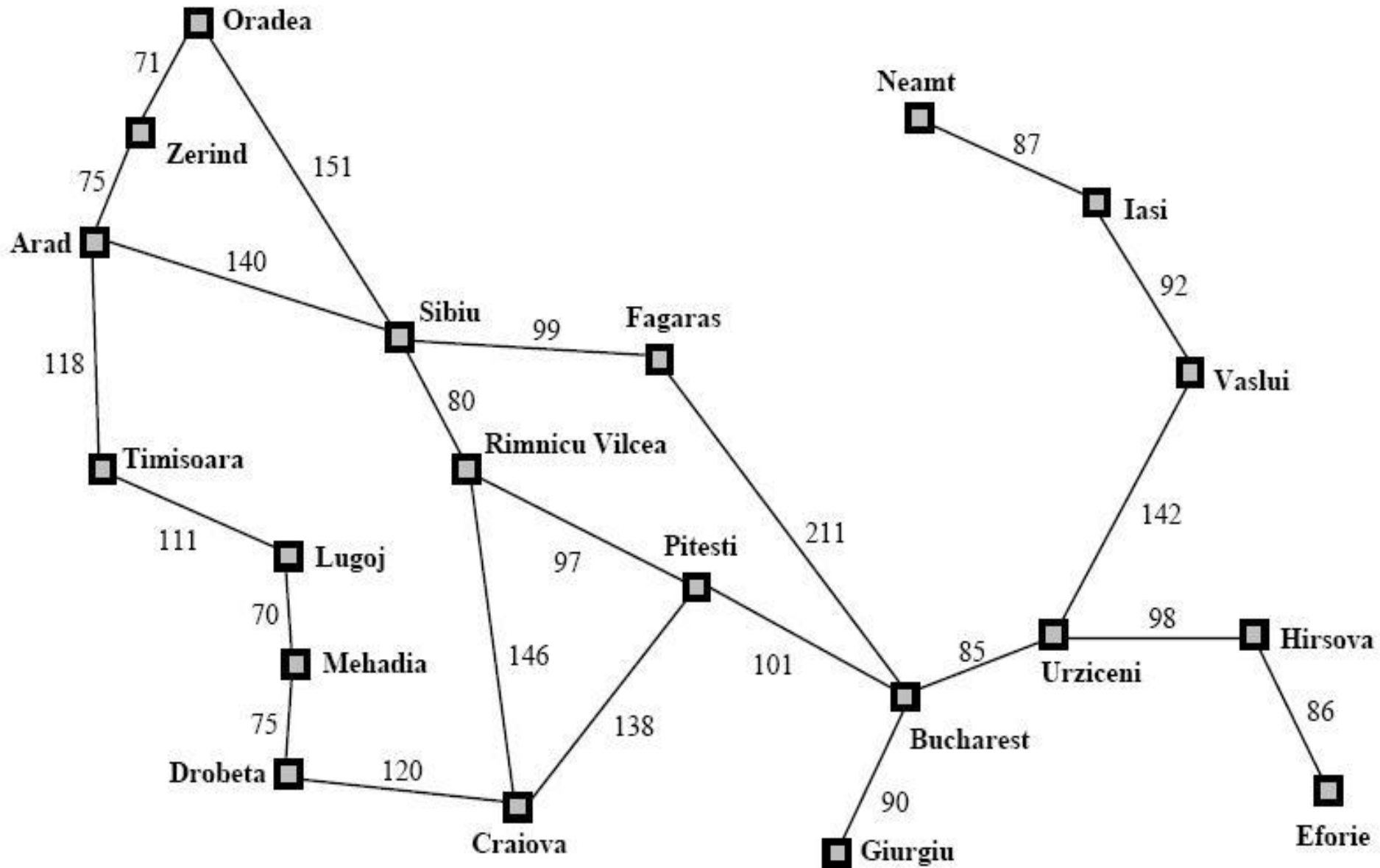
Searching for Solutions

- The initial state + possible action sequences form a **search tree** -- start state as root, branches as actions and nodes as states in state space
- Key steps in a searching process (Growing search tree)
 - Given a state, test if it's the goal state
 - If not, expand current state by applying set of actions
 - generates new set of states (the new leaf nodes generated called frontier)
 - Expand nodes on frontier till solution is found or no more states

Tree Search example



Road map of Romania



Graph Search vs. Tree Search

- Arad $\rightarrow$ Sibiu $\rightarrow$ Arad : Repeated Arad in search tree
- Resulted in a loopy path, a special case of redundant path (when more than 1 path exists to do same thing)
 - Ex: Arad $\rightarrow$ Sibiu (or)
 - Arad $\rightarrow$ Zerind $\rightarrow$ Ordea $\rightarrow$ Sibiu
- **Key idea:** *If idea is to reach goal g that is reachable from a state s , no reason to have redundant paths to s from current state*
- **Reason:** *If path 1 and 2 are paths to s , extending any one of them reaches goal*
- Search algorithms try to eliminate redundant paths for tractability
- Key difference between **Tree Search** and **Graph Search**

Key steps of Graph Search

- Maintain explored set (remembers expanded nodes)
- If a newly generated set already in explored set – Do not add to frontier
- Frontier separates explored vs. unexplored regions
- Hash tables for modeling explored sets – Allows efficient checking for repeated states
- Queue : FIFO – First In First Out
 - Empty(queue) , Pop(queue), Insert(element, queue)
- Stack : LIFO – Last In First Out

Infrastructure for Search Algorithms

- For each node in the tree
 - **State** – Corresponds to current node
 - **Parent** – Node in search tree that generated current node
 - **Action** – Action that generated the node
 - **Path-cost** – Function of step costs that lead from initial state of node (Constructed using parent pointers)
- Function **CHILD-NODE(problem,parent,action)** returns node
 - Return a node with
 - State = problem.Result(parent.State, action)
 - Parent = parent, Action = action
 - Path-cost = parent.Path-cost + problem.Step-cost(parent.State, action)
- Terminology: node is a data structure to represent search tree vs. state is a term used in modeling the world

Measuring Performance

- **Completeness:** Is algorithm guaranteed to find solution ?
- **Optimality:** Does it find the optimal solution ?
- **Time Complexity:** How long it takes to find a solution ?
- **Space Complexity:** How much memory to perform the search ?

- Problem Size : Measured using
 - **b** the branching factor,
 - **d** the depth of shallowest goal node and
 - **m** the maximum length of any path in state space
- Time measured by number of nodes generated during search
- Space by maximum number of nodes stored in memory
- Total cost : Can combine search cost (typically time + maybe memory) + path cost of the solution

Uninformed Search Strategies

(Chapter 3, Section 4)

- Have no additional information apart from problem definition
- Distinguished by order in which nodes are expanded
- If we know one non-goal state is more promising than another – Called informed or heuristic search

- **Breadth first search**
- **Uniform cost search**
- **Depth first search**
- **Depth limited search**
- **Iterative deepening depth first search**
- **Bidirectional search**

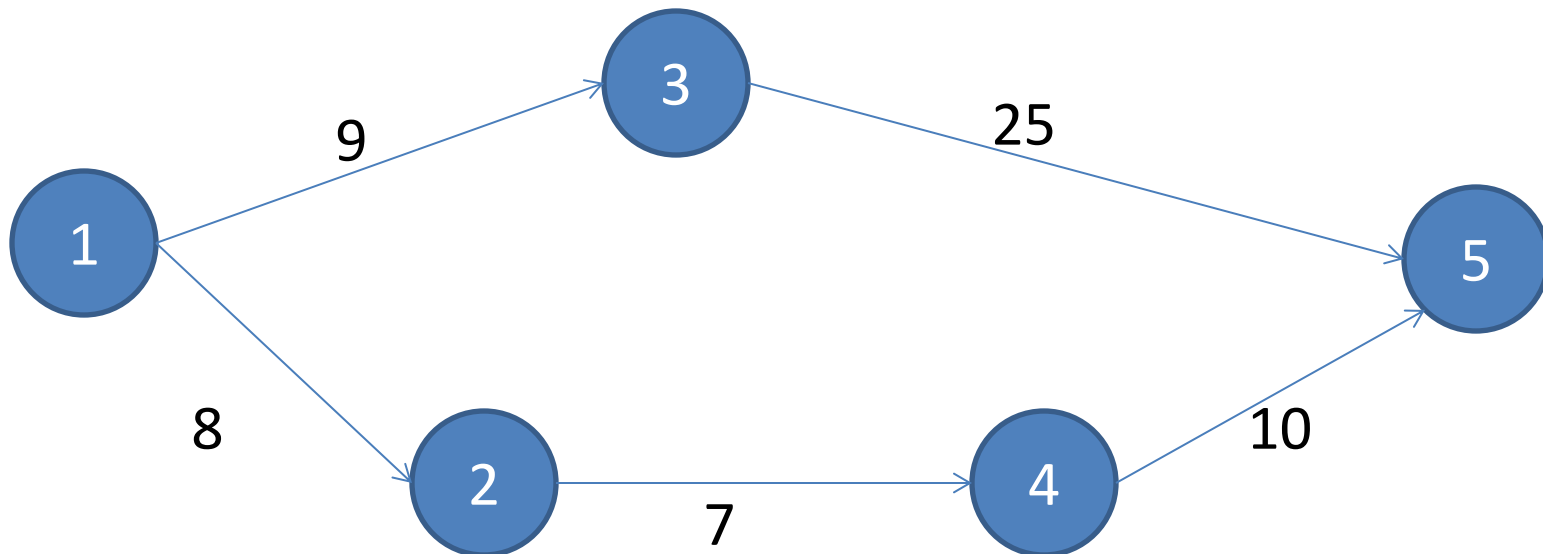
Breadth first search

- **Shallowest unexpanded** node is chosen for expansion (ex: all actions have same cost)
- FIFO queue used
- Deeper nodes always at back of queue
- Goal test applied when a node is generated before adding to queue
- **Is complete:** Eventually all nodes searched
- **Is optimal:** Shallowest goal node always found first, hence optimal if path cost is non-decreasing function of depth of node
- **Time complexity:** $b + b^2 + \dots + b^d = O(b^d)$
- **Space complexity:** Graph search stores all explored nodes $O(b^d)$ (Memory is a big issue)

Breadth first search on a binary tree

Uniform Cost Search

- If step costs non-uniform – Expand node n with the **lowest path cost** (instead of shallowest)
- Order queue by path cost
- Goal test applied when selected **for expansion** (not when generated – Why ??)



Uniform Cost Search

- **Is optimal:** If a node is selected for expansion, optimal path to it has been found
 - Otherwise it would not come up for expansion
- Assumes step costs are **non-negative** i.e. paths never get shorter
- Paths with infinite sequence of **zero-cost actions** leads to infinite loops
 - step cost of at least ϵ assumed
- Guided by path costs rather than depth

Uniform Cost Search

- Time and space complexity $O(b^{1+\lceil \log_b(C^*/\epsilon) \rceil})$
 - C^* is cost of optimal solution
- When **all step costs equal**, similar to breadth-first search except for the Stop Test (costs for priority queue abstracted out here)

Uniform Cost Search Algorithm

Depth First Search

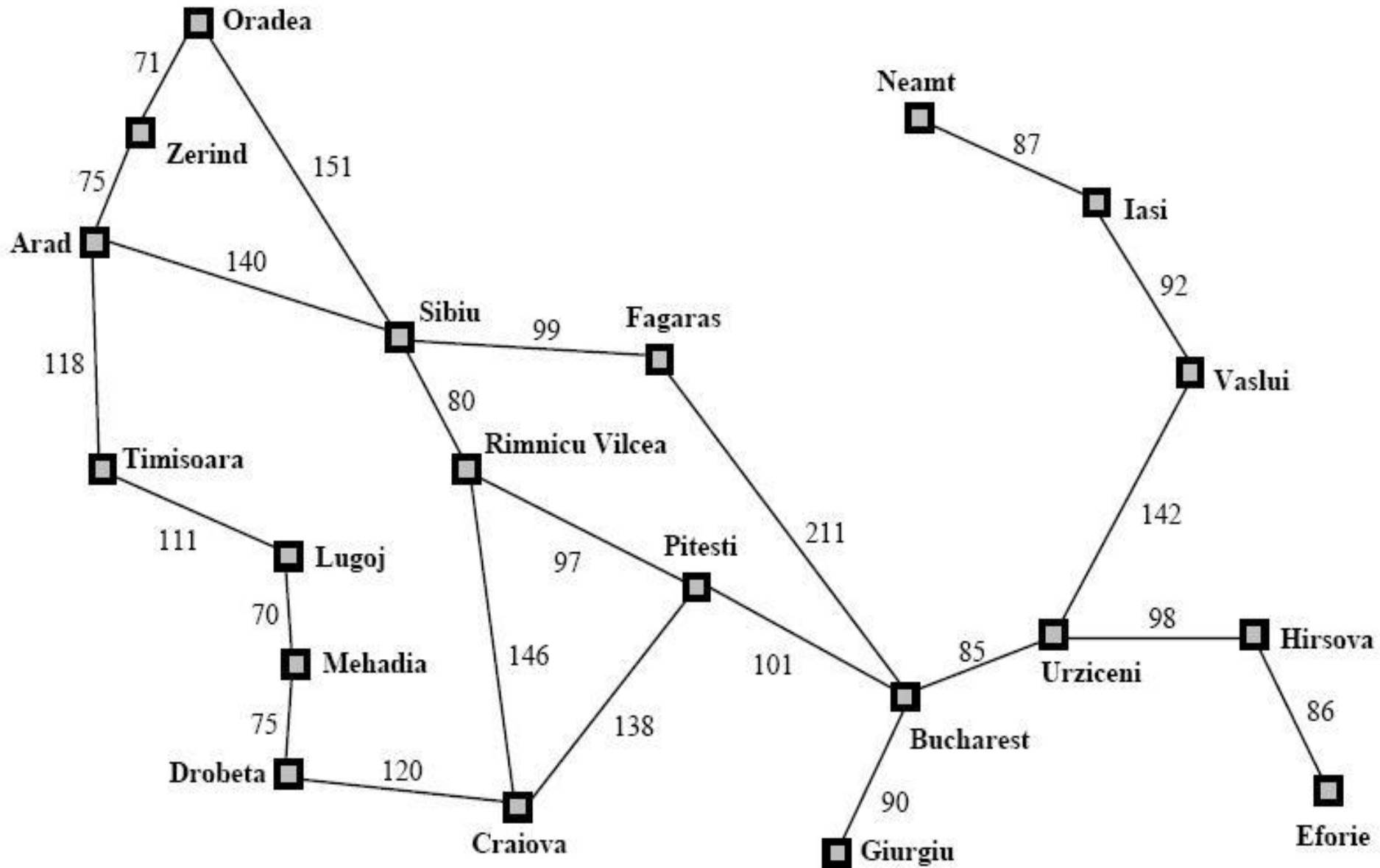
- Expand **deepest** node in the frontier
- LIFO stack used
- Recursive implementations are common (Recursively expands the child)
- Graph search is complete [Tree search can have infinite loops]
 - Check new state against those on path (avoids loops)
 - Does not avoid redundant paths
- Is **non-optimal** (Returns the first encountered goal state)
- **Time complexity** for tree search $O(b^m)$ where m is the maximum depth
- **Space complexity** $O(bm)$ for tree search [for graph search the explored set is still huge]
- Depth first tree search a very popular method due to **very less memory usage**

Depth first search on a binary tree

Depth Limited Search

- Perform depth first search for a **limited depth l**
- Incomplete if $l < d$, non optimal if $l > d$
- $O(b^l)$ time and $O(bl)$ space complexity
- Depth limits can sometimes be picked based on problem
 - Ex: If 20 cities in the Romania map, longest length will be 19 from source
 - In fact, max depth is 9 steps

Road map of Romania



IDS: Iterative Deepening (Depth-First) Search

- General technique usually combined with depth first search
- Iteratively **increase the depth** searched – first 0 (root), then 1, then 2,.... until a goal is found
- **Combines** benefits of depth first and breadth first search
- Memory requirements less like depth first $O(bd)$
- **Complete** like breadth-first and **optimal** when path cost is non-decreasing
- Seems wasteful since same states generated multiple times (**but not actually**)
 - $N(\text{IDS}) = (d)b + (d-1)b^2 + \dots + (1)b^d = O(b^d)$
- Preferred search for large state spaces with unknown depth
- Iterative lengthening search – IDS version for uniform search

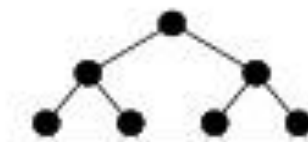
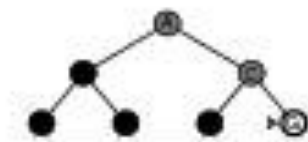
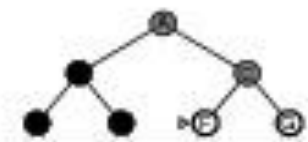
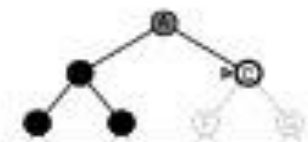
L.lmit = 0



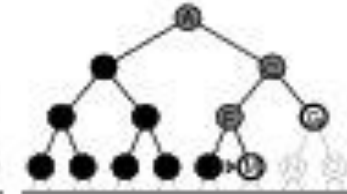
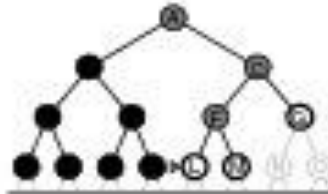
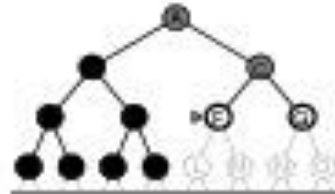
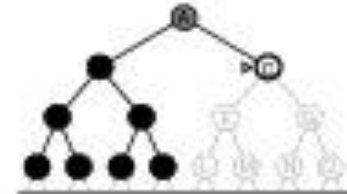
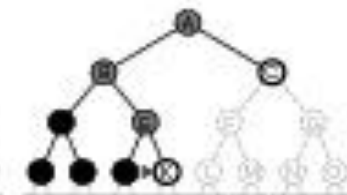
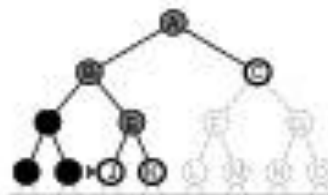
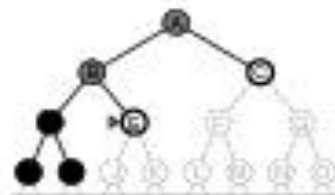
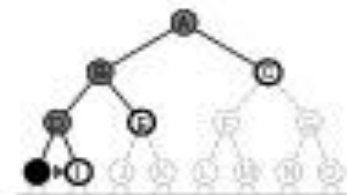
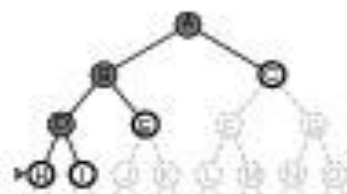
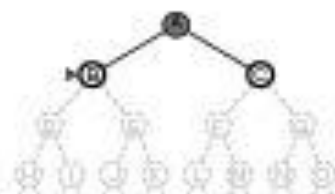
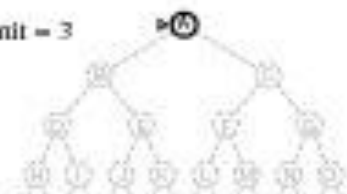
L.lmit = 1



L.lmit = 2



L.lmit = 3



Bidirectional Search

- A **forward and backward** breadth first search simultaneously
- $b^{(d/2)} + b^{(d/2)}$ is much less than b^d
- **$O(b^{d/2})$** time and space complexity
- A solution will be present at the intersection of frontiers (additional checks needed for optimality)
- How to search backwards ?
 - Need to compute predecessors
- If multiple goals, dummy goal state whose immediate predecessors are the goal states
- Search difficult if goal state is abstract ex: no queen attacks another queen